

Guía Visual de Tipos de JOIN en SQL: Dominando la Unión de Datos

Estudiante: Yaretzi Castro Díaz | Grupo: 3DSM-G1.



Tabla Cuentas (Account)

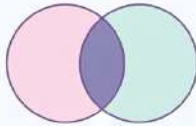
Account: A1, C1
Account: A2, C2
Account: customer_id
Account: ...



Tabla Clientes (Customer)

Customer: ...
Customer1: C1, Nom1
Customer2: C2, Nom2
Customer3: C3, Nom3

Los JOINS permiten combinar registros de dos o más tablas mediante una condición de relación (claves foráneas). Fundamental para reconstruir información y generar análisis.



1. INNER JOIN: El estándar de oro

Solo registros coincidentes. Devuelve únicamente las filas donde existe una coincidencia exacta en ambas tablas.

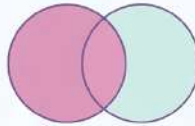
```
SELECT * FROM account  
JOIN customer  
ON account.customer_id =  
customer.customer_id;  
(INNER es opcional)
```

Cuenta		Cliente	
A1	C1	C1	Nom1
A2	C2	C2	Nom2

Resultado Inner

A1	C1	Nom1
A2	C2	Nom2

Solo verás a los clientes con cuentas activas (intersección perfecta).



2. LEFT JOIN: Prioridad a la izquierda

Todo de la izquierda y coincidencias de la derecha. Mantiene todos los registros de la tabla "A" (izq), aunque no haya match.

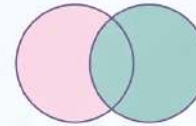
```
SELECT * FROM account  
LEFT JOIN customer  
ON account.customer_id =  
customer.customer_id;
```

Cuenta (Izq)		Cliente (der)	
A1	C1	C1	Nom1
A2	C2	C2	Nom2
A3	C99		

Resultado Left

A1	C1	Nom1
A2	C2	Nom2
A3	C99	NULL

Útil para auditar la entidad principal. Las cuentas corporativas muestran NULL en datos de cliente retail.



3. RIGHT JOIN: Prioridad a la derecha

Todo de la derecha y coincidencias de la izquierda. Mantiene todos los registros de la tabla "B" (der).

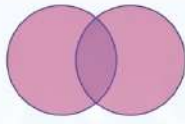
```
SELECT * FROM account  
RIGHT JOIN customer  
ON account.customer_id =  
customer.customer_id;
```

Cuenta (Izq)		Cliente (der)	
A1	C1	C1	Nom1
		C2	Nom2
		C3	Nom3

Resultado Right

A1	C1	Nom1
NULL	C2	Nom2
NULL	C3	Nom3

Permite ver todos los clientes, incluye los sin actividad (sempres de cuenta como NULL).



4. FULL OUTER JOIN: La unión total

Todos los registros de ambas tablas. Devuelve todo; donde no hay match, rellena con NULL.

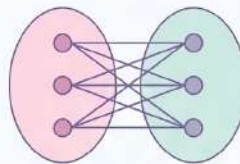
```
SELECT * FROM account  
FULL JOIN customer ON  
account.customer_id = customer_id;  
(Nota técnica: No universal, ej. MySQL requiere UNION)
```

Cuenta		Cliente	
A1	C1	C1	Nom1
A3	C99	C2	Nom2

Resultado Full

A1	C1	Nom1
A3	C99	NULL

Auditoría completa: clientes sin cuentas y cuentas sin clientes registradas.

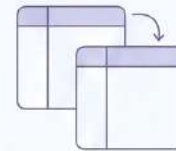


5. CROSS JOIN: Producto Cartesiano

Una cada fila de la primera tabla con todas las filas de la segunda (todas las combinaciones posibles). No usa condición ON.

Matrices de datos / Pruebas.

Tabla A		Tabla B	
1	1	1	X
1	2	1	Y
		2	X
		2	Y



6. SELF JOIN: La tabla ante el espejo

Una tabla se une consigo misma. Requiere alias para diferenciar instancias.

Ej. Empleados y sus supervisores en la misma tabla.

Empleado		Empleado: E1	
E1	Sup: E2	E1	Supervisor: E2
E2	Sup: E3		

Resumen Comparativo Final

Tipo de JOIN	¿Qué devuelve?	¿Genera NULLs?	Uso Principal
INNER	Intersección (Solo coincidencias)	No	Datos relacionados exactos
LEFT	Tabla A completa + coincidencias B	Sí (en lado B)	Identificar huérfanos en B
RIGHT	Tabla B completa + coincidencias A	Sí (en lado A)	Identificar huérfanos en A
FULL	Unión total (Ambas tablas completas)	Sí (en ambos lados)	Auditoría total de datos
CROSS	Todas las combinaciones posibles	No	Análisis combinatorio

Conclusión Técnica: La decisión depende de los datos "huérfanos". Elige INNER JOIN si solo necesitas coincidencias exactas. Elige un OUTER JOIN (Left, Right o Full) si es crítico mantener registros sin pareja.